

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from datetime import timedelta
```

Step 1 (Load Data)

```
cust = pd.read_csv(r"C:\Users\Lenovo\Downloads\
Customer_Master_Data.csv")
```

```
tran = pd.read_csv(r"C:\Users\Lenovo\Downloads\
Customer_Transactions.csv")
```

```
cust.head()
```

	CustomerID	Name	Email	Gender	Age
City \					
0	CUST10000	Onkar Bhargava	pkeer@yahoo.com	Male	54
Delhi					
1	CUST10001	Divit Kohli	mkalita@sarin.com	Female	48
Kolkata					
2	CUST10002	Kiara Behl	apteanay@hotmail.com	Male	75
Kolkata					
3	CUST10003	Vaibhav Sankar	bseshadri@choudhry.info	Male	62
Pune					
4	CUST10004	Shray D'Alia	bdhillon@toor-mall.com	Male	55
Delhi					

	MaritalStatus	NumChildren	JoinDate
0	Divorced	0	2021-02-22
1	Married	0	2023-12-06
2	Widowed	2	2023-08-23
3	Divorced	2	2022-11-17
4	Divorced	0	2022-12-04

```
tran.head()
```

	CustomerID	TransactionDate	TransactionAmount
0	CUST10771	7/31/23	2383.07
1	CUST10100	3/10/24	497.54
2	CUST10031	2/17/25	536.78
3	CUST10987	7/17/23	314.89
4	CUST10831	12/15/24	2543.19

```
cust.shape
```

```
(1000, 9)
```

```
tran.shape
```

```
(23050, 3)

cust.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 9 columns):
#   Column                Non-Null Count  Dtype
---  -
0   CustomerID            1000 non-null   object
1   Name                  1000 non-null   object
2   Email                 1000 non-null   object
3   Gender                1000 non-null   object
4   Age                   1000 non-null   int64
5   City                  1000 non-null   object
6   MaritalStatus         1000 non-null   object
7   NumChildren           1000 non-null   int64
8   JoinDate              1000 non-null   object
dtypes: int64(2), object(7)
memory usage: 70.4+ KB

tran.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 23050 entries, 0 to 23049
Data columns (total 3 columns):
#   Column                Non-Null Count  Dtype
---  -
0   CustomerID            23050 non-null   object
1   TransactionDate        23050 non-null   object
2   TransactionAmount      23050 non-null   float64
dtypes: float64(1), object(2)
memory usage: 540.4+ KB
```

Step 2 (Clean Data)

```
tran['TransactionDate'] = pd.to_datetime(tran['TransactionDate'])
cust['JoinDate'] = pd.to_datetime(cust['JoinDate'])

C:\Users\Lenovo\AppData\Local\Temp\ipykernel_2968\327917176.py:1:
UserWarning: Could not infer format, so each element will be parsed
individually, falling back to `dateutil`. To ensure parsing is
consistent and as-expected, please specify a format.
    tran['TransactionDate'] = pd.to_datetime(tran['TransactionDate'])

cust.isnull().sum()

CustomerID    0
Name          0
Email         0
Gender        0
```

```

Age          0
City          0
MaritalStatus 0
NumChildren  0
JoinDate      0
dtype: int64

tran.isnull().sum()

CustomerID    0
TransactionDate 0
TransactionAmount 0
dtype: int64

cust['CustomerID'].duplicated().sum()

np.int64(0)

```

Step 3 (Merge)

```

df = pd.merge(tran, cust, on='CustomerID', how='left')
df.head()

```

	CustomerID	TransactionDate	TransactionAmount	Name \
0	CUST10771	2023-07-31	2383.07	Lakshay Dhillon
1	CUST10100	2024-03-10	497.54	Aniruddh Borah
2	CUST10031	2025-02-17	536.78	Ritvik Ahuja
3	CUST10987	2023-07-17	314.89	Jayan Wagle
4	CUST10831	2024-12-15	2543.19	Ishita Agarwal

	Email	Gender	Age	City
MaritalStatus \				
0	dharmajantara@gmail.com	Female	27	Ahmedabad
Widowed				
1	jivikabhavsar@gmail.com	Female	53	Lucknow
Divorced				
2	jhaverifarhan@chandra.org	Male	40	Ahmedabad
Single				
3	ojas82@gmail.com	Not Disclosed	75	Bangalore
Widowed				
4	vbalay@yahoo.com	Not Disclosed	27	Jaipur
Divorced				

	NumChildren	JoinDate
0	3	2023-05-31
1	4	2022-01-08
2	0	2023-12-03
3	4	2022-03-18
4	4	2021-06-30

Step 4 (RFM Calculation)

```
LastDate = df.groupby('CustomerID')
['TransactionDate'].max().reset_index().rename(columns={'TransactionDate': 'LastTransactionDate'})
```

LastDate

	CustomerID	LastTransactionDate
0	CUST10000	2025-07-17
1	CUST10001	2025-06-25
2	CUST10002	2025-07-12
3	CUST10003	2025-05-10
4	CUST10004	2025-07-22
...	...	...
995	CUST10995	2024-06-23
996	CUST10996	2025-07-15
997	CUST10997	2025-06-28
998	CUST10998	2025-03-26
999	CUST10999	2025-07-25

[1000 rows x 2 columns]

```
today = df['TransactionDate'].max() + timedelta(days=1)
```

today

```
Timestamp('2025-07-30 00:00:00')
```

```
cust['Recency'] = (today - LastDate['LastTransactionDate']).dt.days
```

```
cust['Recency'].value_counts()
```

Recency

1	28
11	26
7	23
4	23
3	21

...

128	1
274	1
427	1
202	1
129	1

Name: count, Length: 184, dtype: int64

```
cust['Recency'].isnull().sum()
```

```
np.int64(0)
```

```
Frequency = df.groupby('CustomerID')
['TransactionDate'].count().reset_index().rename(columns={'Transaction
Date': 'Frequency'})
```

```
Monetary = df.groupby('CustomerID')
['TransactionAmount'].sum().reset_index().rename(columns={'Transaction
Amount': 'Monetary'})
```

```
df_rfm = cust[['CustomerID', 'Recency']].merge(Frequency,
on='CustomerID', how='left')
```

df_rfm

	CustomerID	Recency	Frequency
0	CUST10000	13	23
1	CUST10001	35	30
2	CUST10002	18	24
3	CUST10003	81	25
4	CUST10004	8	19
...	...	...	...
995	CUST10995	402	21
996	CUST10996	15	21
997	CUST10997	32	20
998	CUST10998	126	25
999	CUST10999	5	23

[1000 rows x 3 columns]

```
df_rfm = pd.merge(df_rfm, Monetary, on='CustomerID', how='left')
```

df_rfm

	CustomerID	Recency	Frequency	Monetary
0	CUST10000	13	23	21265.49
1	CUST10001	35	30	28654.31
2	CUST10002	18	24	23884.03
3	CUST10003	81	25	24206.03
4	CUST10004	8	19	25565.30
...	...	...	...	...
995	CUST10995	402	21	24325.19
996	CUST10996	15	21	21809.11
997	CUST10997	32	20	21120.48
998	CUST10998	126	25	29494.56
999	CUST10999	5	23	22028.01

[1000 rows x 4 columns]

Step 5 (RFM Scoring)

```
df_rfm['R_Score'] = pd.qcut(df_rfm['Recency'], 5, labels=[5, 4, 3, 2,
1])
```

```
df_rfm['F_Score'] = pd.qcut(df_rfm['Frequency'], 5, labels=[1, 2, 3, 4, 5])
df_rfm['M_Score'] = pd.qcut(df_rfm['Monetary'], 5, labels=[1, 2, 3, 4, 5])
```

```
df_rfm
```

	CustomerID	Recency	Frequency	Monetary	R_Score	F_Score	M_Score
0	CUST10000	13	23	21265.49	4	3	2
1	CUST10001	35	30	28654.31	3	5	5
2	CUST10002	18	24	23884.03	4	3	3
3	CUST10003	81	25	24206.03	1	4	3
4	CUST10004	8	19	25565.30	5	1	4
...	...	...	...	...	...	...	...
995	CUST10995	402	21	24325.19	1	2	3
996	CUST10996	15	21	21809.11	4	2	3
997	CUST10997	32	20	21120.48	3	2	2
998	CUST10998	126	25	29494.56	1	4	5
999	CUST10999	5	23	22028.01	5	3	3

```
[1000 rows x 7 columns]
```

Step 6 (Segment Code)

```
df_rfm['Segment'] = df_rfm['R_Score'].astype(str) +
df_rfm['F_Score'].astype(str) + df_rfm['M_Score'].astype(str)
```

```
df_rfm.head()
```

	CustomerID	Recency	Frequency	Monetary	R_Score	F_Score	M_Score	Segment
0	CUST10000	13	23	21265.49	4	3	2	432
1	CUST10001	35	30	28654.31	3	5	5	355
2	CUST10002	18	24	23884.03	4	3	3	433
3	CUST10003	81	25	24206.03	1	4	3	143
4	CUST10004	8	19	25565.30	5	1	4	514

Step 7 (Segment Labels)

```
df_rfm['Segment_Labels'] = 'others'
```

```
df_rfm.loc[df_rfm['Segment'] == '555', 'Segment_Labels'] = 'Champions'
df_rfm.loc[df_rfm['Segment'] == '111', 'Segment_Labels'] = 'Lost'
```

```

df_rfm.loc[(df_rfm['Segment'].str[0] == '5') & (df_rfm['Segment'] !=
'555'), 'Segment_Labels'] = 'Recent'

df_rfm.loc[(df_rfm['Segment'].str[1] == '5') & (df_rfm['Segment'] !=
'555'), 'Segment_Labels'] = 'Frequent'

df_rfm.loc[(df_rfm['Segment'].str[2] == '5') & (df_rfm['Segment'] !=
'555'), 'Segment_Labels'] = 'Big Spenders'

df_rfm['Segment_Labels'].value_counts()

Segment_Labels
others          543
Big Spenders    165
Recent          160
Frequent         63
Champions        35
Lost             34
Name: count, dtype: int64

df_rfm.head()

```

	CustomerID	Recency	Frequency	Monetary	R_Score	F_Score	M_Score
0	CUST10000	13	23	21265.49	4	3	2
1	CUST10001	35	30	28654.31	3	5	5
2	CUST10002	18	24	23884.03	4	3	3
3	CUST10003	81	25	24206.03	1	4	3
4	CUST10004	8	19	25565.30	5	1	4

```

Segment_Labels
0      others
1  Big Spenders
2      others
3      others
4      Recent

```

Step 8 (Visualizations)

Plot 1: Bar chart — Customer Segment Distribution

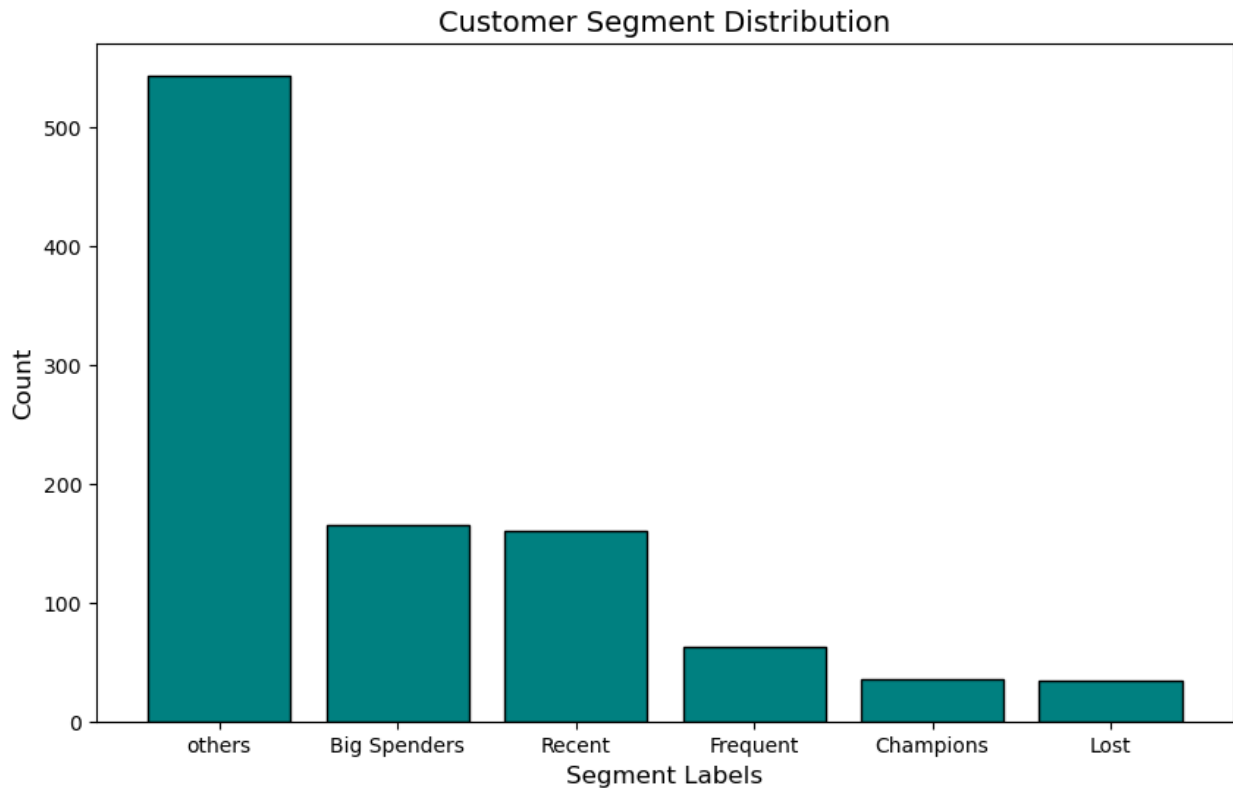
```

index = df_rfm['Segment_Labels'].value_counts().index
values = df_rfm['Segment_Labels'].value_counts().values

plt.figure(figsize=(10, 6))

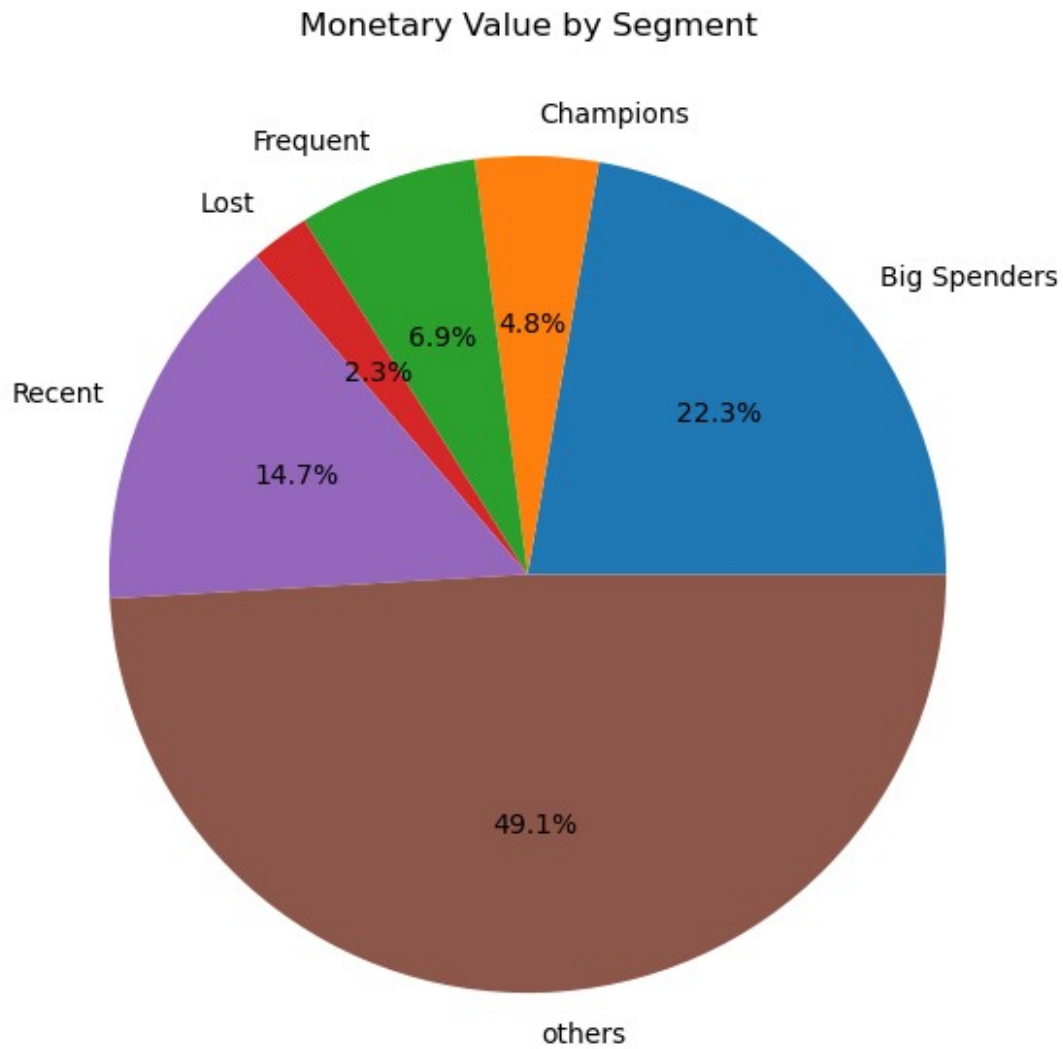
```

```
plt.bar(index, values, color='teal', edgecolor='black')
plt.title('Customer Segment Distribution', fontsize=14)
plt.xlabel('Segment Labels', fontsize=12)
plt.ylabel('Count', fontsize=12)
plt.show()
```



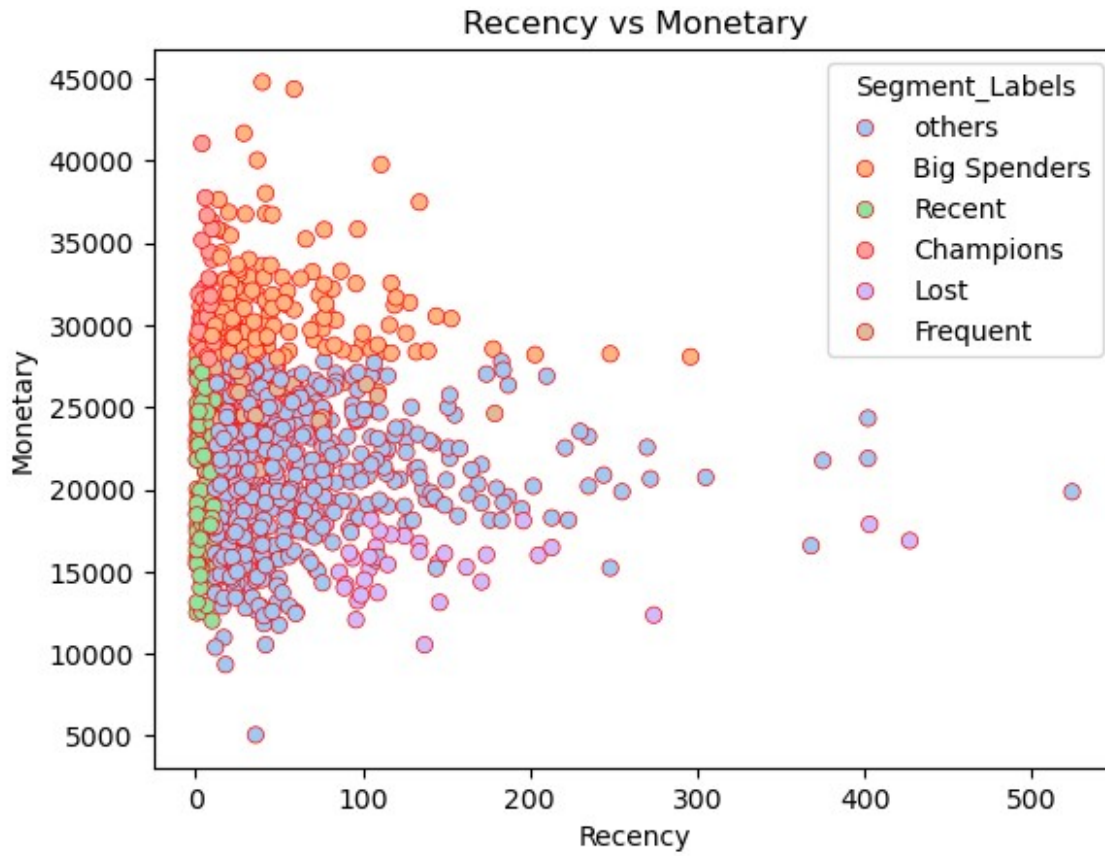
Plot 2: Pie chart — Monetary Value by Segment

```
df_rfm.groupby('Segment_Labels')['Monetary'].sum().plot(
    kind='pie',
    autopct="%0.1f%%",
    figsize=(7, 7),
    ylabel='',
)
plt.title("Monetary Value by Segment")
plt.show()
```

Plot 3: Scatter plot — Recency vs Monetary

```
sns.scatterplot(x='Recency', y='Monetary', data=df_rfm,  
hue='Segment_Labels', palette='pastel', edgecolor='red')  
plt.title('Recency vs Monetary')  
plt.show()
```



Pareto Analysis

```
df = df_rfm[['CustomerID', 'Monetary']]
df = df.sort_values(by='Monetary', ascending=False)
df['Cumulative_Revenue'] = df['Monetary'].cumsum()
df['Revenue_Percent'] = df['Cumulative_Revenue'] /
df['Monetary'].sum() * 100
df = df.reset_index(drop=True)
df['Customer_Percent'] = (df.index + 1) / len(df) * 100
```

See what % revenue the top 20% customers contribute

```
df[df['Customer_Percent'] <= 20].tail(1)
```

	CustomerID	Monetary	Cumulative_Revenue	Revenue_Percent	\
199	CUST10338	27838.98	6231163.35	27.029495	
	Customer_Percent				
199		20.0			

```
df.head()
```

	CustomerID	Monetary	Cumulative_Revenue	Revenue_Percent
0	CUST10944	44784.99	44784.99	0.194268
1	CUST10510	44367.33	89152.32	0.386724
2	CUST10053	41674.56	130826.88	0.567500
3	CUST10776	41050.76	171877.64	0.745570
4	CUST10696	40035.48	211913.12	0.919235

Checking that if it really follows pareto analysis

```
top20_revenue = df[df['Customer_Percent'] <= 20]
['Revenue_Percent'].iloc[-1]
print(f"Top 20% of customers contribute {top20_revenue:.1f}% of total revenue")

if top20_revenue >= 80:
    print("This FOLLOWS the Pareto 80-20 rule")
else:
    print(f"This does NOT follow the Pareto rule")
```

Top 20% of customers contribute 27.0% of total revenue
This does NOT follow the Pareto rule

Pareto Graph

```
plt.figure(figsize=(10, 5))
plt.plot(df['Customer_Percent'], df['Revenue_Percent'],
color='steelblue', linewidth=2)
plt.axvline(x=20, color='red', linestyle='--', label='Top 20% Customers')
plt.axhline(y=top20_revenue, color='green', linestyle='--',
label=f'{top20_revenue:.1f}% Revenue')
plt.title('Pareto Analysis: Customer % vs Revenue %', fontsize=14)
plt.xlabel('Customer %')
plt.ylabel('Revenue %')
plt.legend()
plt.grid(alpha=0.3)
plt.show()
```

